
Web Application Requirements Specification and Analysis App

Sprint Report

Δρίβας Ιωάννης, 5216

Κολονέλου Ειρήνη, 5456

Παππά Ευγενία, 5534

VERSIONS HISTORY

Date	Version	Description	Author
25/4	1.0	Development of the first user stories for user registration and login to the system, as well as the initial setup of the database	Ioannis Drivas, Kolonelo u Eirini, Pappa Eugenia
2/5	1.1	Development of the creation, viewing, managing and deletion of projects from the user's account	Ioannis Drivas, Kolonelo u Eirini, Pappa Eugenia
8/5	1.2	Implementation of use case and CRC card management, including their creation, deletion, as well as the establishment of links between them	Ioannis Drivas, Kolonelo u Eirini, Pappa Eugenia
16/5	1.3	Implementation of the automatic generation of use case and CRC class diagrams, producing scripts that can be visualized using Nomnoml	Ioannis Drivas, Kolonelo u Eirini, Pappa Eugenia
23/5	2.0	Preparation of the project report and development of the UML diagrams	Ioannis Drivas, Kolonelo u Eirini, Pappa Eugenia

1 Introduction

This document provides information concerning the project.

1.1 Purpose

The purpose of this project is to develop a web-based application that supports software developers in the specification and analysis of requirements for their software engineering projects. The application enables developers to create and manage their own projects, organizing all related requirements and designing artifacts in one place. Within each project, developers can define use cases by specifying actors, preconditions, main flow of events, alternative flows, and post conditions. For requirements analysis, the application allows developers to create and manage CRC (Class-Responsibility-Collaboration) cards, where each card defines a class name, its responsibilities, and the other classes it collaborates with. Furthermore, developers can link CRC cards to use cases to show how project classes support the use case behavior. Additionally, the application provides diagram generation capabilities, allowing developers to automatically generate scripts that can be used by text-based UML tools such as Nomnoml to visualize use case diagrams and class diagrams. The application follows a layered architecture that promotes low coupling and high cohesion, ensuring maintainability and extensibility.

1.2 Document Structure

The rest of this document is structured as follows. Section 2 describes our Scrum team and specifies the Sprint's backlog. Section 3 specifies the main design concepts for this release of the project.

2 Scrum team and Sprint Backlog

2.1 Scrum team

Product Owner	Zarras Apostolos
Scrum Master	Drivas Ioannis
Development Team	Drivas Ioannis, Kolonelou Eirini, Pappa Eygenia

2.2 Sprints

Sprint No	Begin Date	End Date	Number of weeks	User stories
1	25/4	1/5	1	US1,US2,US3
2	2/5	8/5	1	US4,US5,US6
3	8/5	15/5	1	US7,US8,US9,US10,US11,US12,US13,US14
4	16/5	24/5	1.5	US15,US16

3 Use Cases

3.1 <User Register>

Use case ID	UC1
Actors	User
Pre conditions	User is not registered to the app.
Main flow of events	1. The use case starts when the user selects "Register" 2. The user enters required details (username, password). 3. The system validates the details and creates a new account. 4. The user can now select "Login" to log into the app.
Alternative flow 1	User already exists. 1. The user is redirected to the login page
Post conditions	User now exists in the SQL database and can log into the app.

3.2 <User Login>

Use case ID	UC2
Actors	User
Pre conditions	User already has an existing account.

Main flow of events	1. The use case starts when the user selects “Login” 2. The user enters required details (username, password). 3. The user clicks the “Login” button. 4. The users is redirected to the User Dashboard.
Alternative flow 1	The user enters invalid credentials. 1. The user is redirected to the same login page. 2. The system displays an error message “Username or Password is invalid.”.
Post conditions	User is logged in and can access functionalities.

3.3 <User Logout>

Use case ID	UC3
Actors	User
Pre conditions	User is logged in.
Main flow of events	1. The use case starts when the user selects "Logout." 2. The user is redirected to the login page.
Post conditions	User session is terminated, and access to functionalities is revoked.

3.4 <Manage Profile>

Use case ID	UC4
Actors	User
Pre conditions	User is logged in.
Main flow of events	1. The use case starts when the user selects “Edit Profile”. 2. The user enters his personal details(Full name and E-mail, new password) 3.The user clicks the “save” button.
Alternative	The user selects “cancel”

flow 1	1.The user is redirected to the user dashboard
Post conditions	User details are now saved in the SQL database and will appear anytime they access their profile.

3.5 <View Project>

Use case ID	UC5
Actors	User
Pre conditions	User is logged in and in the User dashboard page.
Main flow of events	1.The use case starts when the user selects “View Project” 2.The user is redirected to the Project Dashboard and can view their already existing projects.
Alternative flow 1	The user selects “Back to the User Dashboard” 1.The user is redirected to the user dashboard.
Post conditions	The user views their existing projects.

3.6 <Create new Project>

Use case ID	UC6
Actors	User
Pre conditions	User is logged in and in the Project dashboard page.
Main flow of events	1.The use case starts when the user selects “create Project”. 2.The user is redirected to the Create new project page. 3..The user enters the name of the new project 4.The user clicks the “save” button.
Alternative flow 1	The user selects “Back to the Project dashboard ” 1.The user is redirected to the project dashboard
Post conditions	The new project is now saved in the SQL database and will appear anytime they access their profile.

3.7 <Delete Project>

Use case ID	UC7
Actors	User
Pre conditions	User is logged in and in the Project dashboard page.
Main flow of events	1.The use case starts when the user selects “delete” in an already existing project in the Project dashboard. 2. System displays a confirmation message "Delete this project?" 2.1 The user selects “OK” 2.1.1 The projects is now deleted. 2.2 The user selects “Ακύρωση” 2.2.1 Nothing happens.
Post conditions	The project is now deleted from the SQL database and will not longer appear anytime the user access their profile.

3.8 <Define Use Cases>

Use case ID	UC8
Actors	User
Pre conditions	User is logged in and in a specific’s project page.
Main flow of events	1.The use case starts when the user selects “Add” in the use case table. 2.The user is redirected to the “Create New Use Case” page. 3..The user enters the Use Case Name, the Actor , Preconditions , Main Flow and Post conditions 4.The user clicks the “save” button.
Alternative flow 1	The user selects “Back” 1.The user is redirected to the project dashboard
Post conditions	The new use case is now saved in the SQL database and will appear anytime they access their project.

3.9 <Update Use Case>

Use case ID	UC9
Actors	User
Pre conditions	User is logged in and in a specific's Use Case page in the selected project.
Main flow of events	1. The use case starts when the user selects "View" in a use case. 2. The user can update(edit or clear) the details for the use case (actors, preconditions, main flow and post conditions) 2.1 The user clicks "Edit" in a detail of the Use Case. 2.1.1 The user changes the details. 2.1.1.1 The user clicks "save". 2.1.1.1 The changes are applied in the use case details 2.1.1.2 The user clicks "cancel". 2.1.1.2.1 Nothing changes. 2.1.2 The user clicks "clear" 2.1.2.1 System displays a confirmation message "Clear this field?" 2.1.2.1.1 The user clicks "Ok" 2.1.2.1.1.1 The field is now cleared. 2.1.2.1.2 The user clicks "Ακύρωση" 2.1.2.1.2.1 Nothing happens 3.The user writes a description for an alternative flow and clicks on "+Add" 3.1 The alternative flow is now saved for this use case 3.1.1 The user selects "Delete" 3.1.1.1 System displays a confirmation message "Delete this flow?" 3.1.1.1 The user clicks "OK" 3.1.2.1 The user clicks "Ακύρωση" 3.1.2.1.1 Nothing happens
Alternative flow 1	The user selects "Back to the Project " 1.The user is redirected to the project dashboard
Post conditions	The updated use case is now saved in the SQL database and will appear anytime they access their profile.

3.10 <View list of the Use Cases>

Use case ID	UC10
Actors	User
Pre conditions	User is logged in and in a specific's project page.
Main flow of events	1. The use case starts when the user selects a specific project name. 2. The user sees a list of all the use cases of the project in the "Use Case" table.
Alternative flow 1	The user selects "Back to the Projects " 1.The user is redirected to the project dashboard
Post conditions	The user views the existing use cases of a project.

3.11 <Delete Use Case>

Use case ID	UC11
Actors	User
Pre conditions	User is logged in and in a specific's project page.
Main flow of events	1. The use case starts when the user selects "delete" in a use case of the specific project in the "Use Cases" table.
Alternative flow 1	The user selects "Back to the Projects " 1.The user is redirected to the project dashboard
Post conditions	The use case is now deleted from the SQL database and will not longer appear anytime the user accesses their project.

3.12 <Create CRC Card>

Use case ID	UC12
Actors	User
Pre	User is logged in and in a specific's project page.

conditions	
Main flow of events	1.The use case starts when the user selects “Add” in the “CRC cards” table. 2.The user is redirected to the “Create new CRC Card” page. 3..The user enters the CRC Class Name, the Responsibilities and Collaborations 4.The user clicks the “save” button.
Alternative flow 1	The user selects “Back ” 1.The user is redirected to the project dashboard
Post conditions	The new CRC card is now saved to the SQL database and will appear anytime the user access their project.

3.13 <Update CRC Cards>

Use case ID	UC13
Actors	User
Pre conditions	User is logged in and in a specific’s CRC card page in the selected project.
Main flow of events	1. The use case starts when the user selects “View” in a CRC card. 2. The user can update(edit or clear) the details of the CRC card(Class Name ,Responsibilities and Collaborations) 2.1 The user clicks “Edit” in a Detail of the CRC card. 2.1.1 The user changes the details. 2.1.1.1 The user clicks “save”. 2.1.1.1 The changes are applied in the CRC card details. 2.1.1.2 The user clicks “cancel”. 2.1.1.2.1 Nothing changes. 2.1.2 The user clicks “clear” 2.1.2.1 System displays a confirmation message "Clear this field?" 2.1.2.1.1 The user clicks “Ok” 2.1.2.1.1.1 The field is now cleared. 2.1.2.1.2 The user clicks “Ακύρωση” 2.1.2.1.2.1 Nothing happens

Alternative flow 1	The user selects “Back to the Project ” 1.The user is redirected to the project dashboard
Post conditions	The updated CRC card is now saved in the SQL database and will appear anytime they access their project.

3.14 <Link CRC card to Use Cases>

Use case ID	UC14
Actors	User
Pre conditions	User is logged in and in a specific’s use case page
Main flow of events	1. The user scrolls down to the “Linked CRC Cards” table. 2.The user selects a existing CRC card to the use case from the stack 3.The user clicks on the +Link button.
Alternative flow 1	1.The user selects “Unlink” 1.1 System displays a confirmation message "Unlink this CRC card?" 1.1.1 The user clicks “Ok” 1.1.1.1 The use case is now unlinked from the crc card. 1.2.1 The user clicks “Ακύρωση” 1.2.1.1 Nothing happens 2.The user selects “Back to Project” without linking any crc cards to use cases 2.1 Nothing changes
Post conditions	The Use Case is now linked to the CRC card the user selected.

3.15 <Delete CRC card>

Use case ID	UC15
Actors	User
Pre conditions	User is logged in and in a specific’s use case page.
Main flow of	1. The use case starts when the user selects “delete” in a CRC card of the

events	specific project in the “CRC card” table. 2. System displays a confirmation message “Delete this CRC card?” 2.1 The user clicks “OK” 2.1.1 The crc card is now deleted. 2.2 The user clicks “Ακύρωση” 2.2.1 Nothing happens.
Alternative flow 1	The user selects “Back to the Project ” 1.The user is redirected to the project dashboard
Post conditions	The CRC card is now deleted from the SQL database and will not longer appear anytime the user access their project.

3.16 <Generate Use Case Diagram>

Use case ID	UC16
Actors	User
Pre conditions	User is logged in and in a specific’s project page
Main flow of events	1. The use case starts when the user clicks to the Diagram Generation table and selects “Use Case Diagram”. 2.The user is redirected to the Nomnoml Script so that the user can click on the URL and generate their visualization for their use case
Alternative flow 1	The user selects “Back to the Project ” 1.The user is redirected to the project dashboard
Post conditions	The users views the script of the Use case diagram and is encouraged to copy and paste the script into the nomnoml.com editor to generate the visualization of the diagram

3.17 <Generate CRC card Diagram>

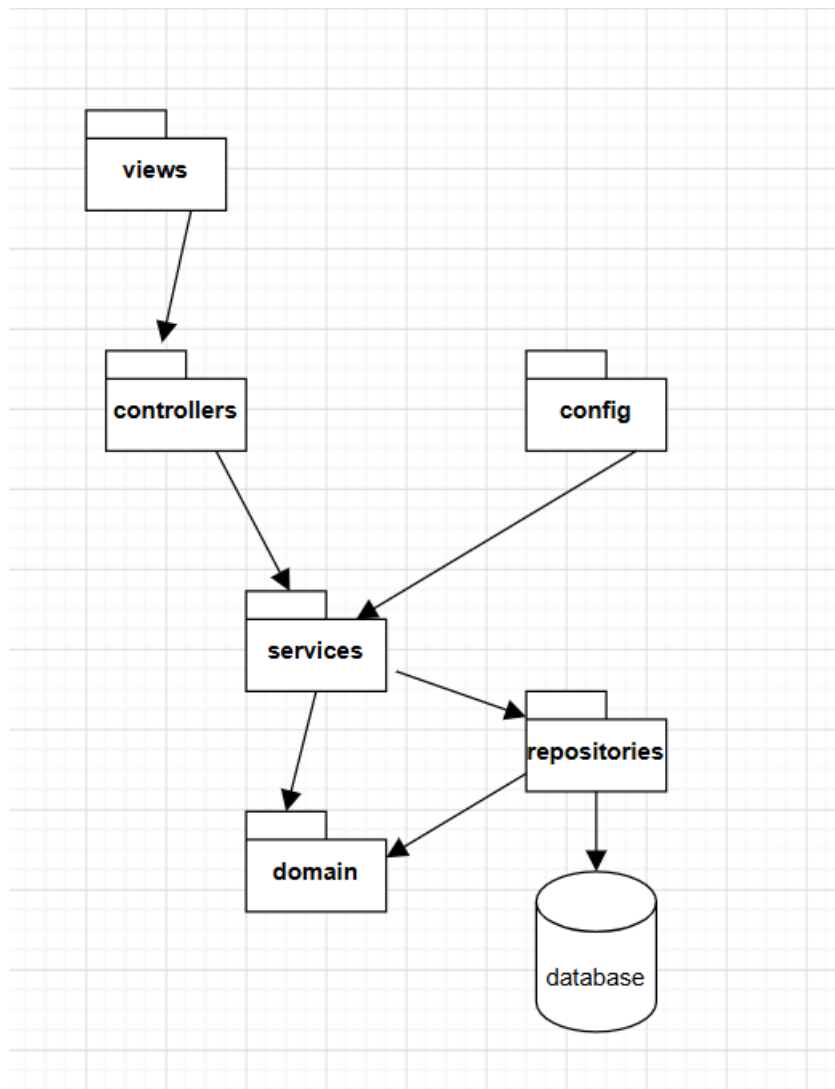
Use case ID	UC16
Actors	User
Pre conditions	User is logged in and in a specific’s project page

Main flow of events	1. The use case starts when the user clicks on the Diagram Generation table and selects "Crc Class Diagram". 2. The user is redirected to the Nomnoml Script so that the user can click on the URL and generate their visualization for their crc card
Alternative flow 1	The user selects "Back to the Project " 1.The user is redirected to the project dashboard
Post conditions	The users views the script of the Crc card diagram and is encouraged to copy and paste the script into the nomnoml.com editor to generate the visualization of the diagram

4 Design

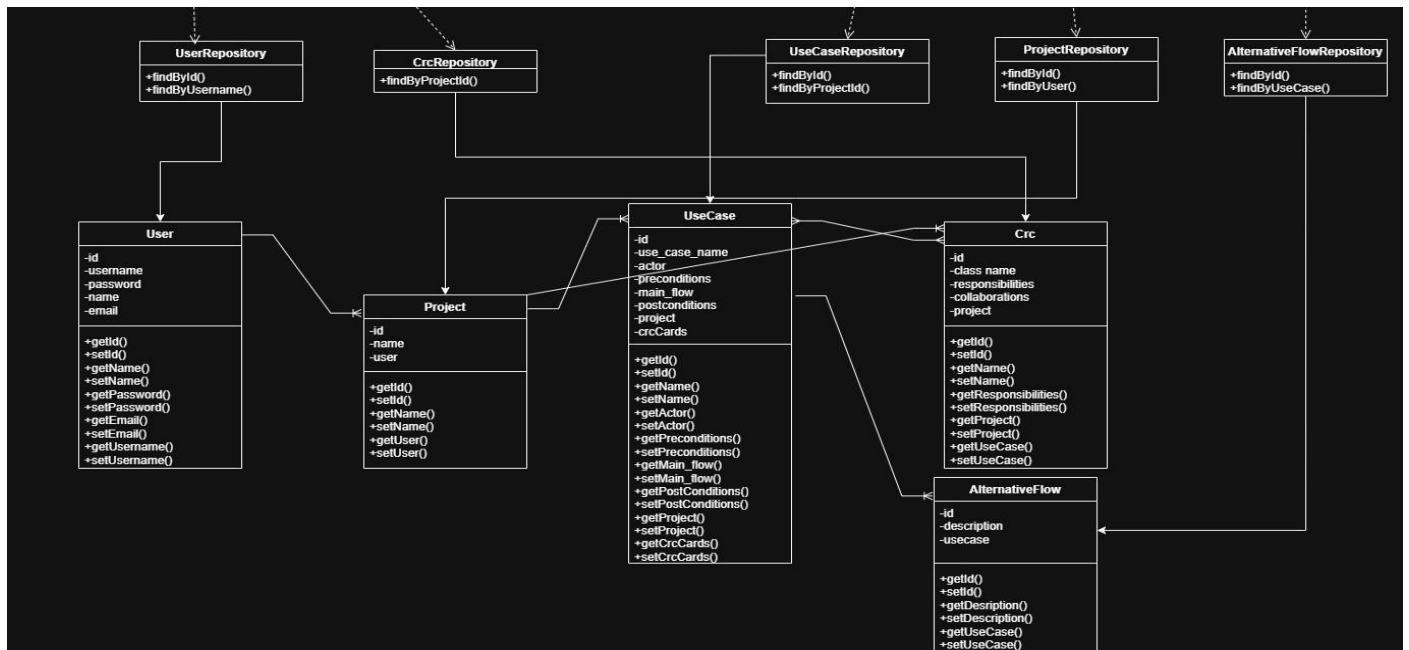
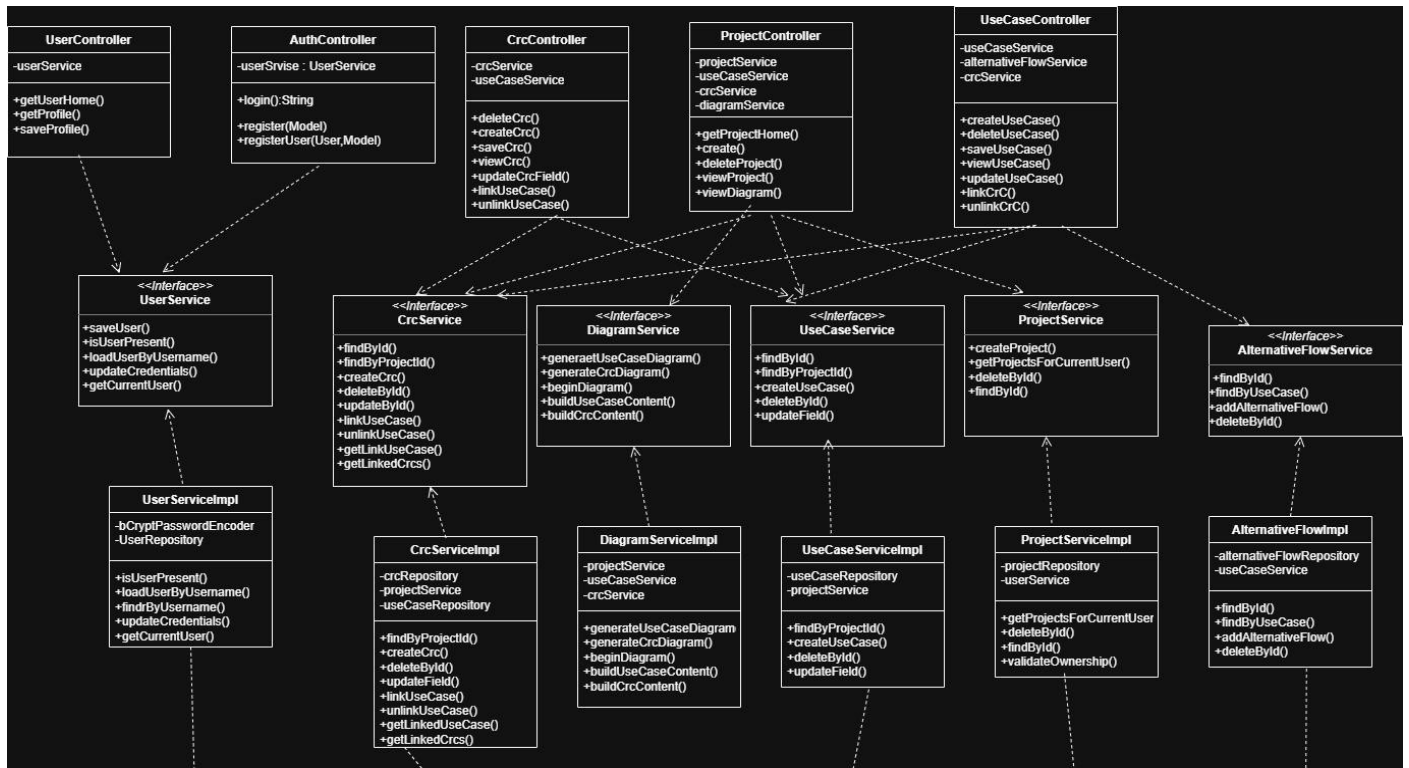
4.1 Architecture

UML package diagram



4.2 Design

UML class diagram



CRC Cards for All System Classes

Controllers

Class Name: AuthController	
Responsibilities: ▪ Handle login ▪ Handle registration ▪ Save new user	Collaborations: ▪ UserService ▪ User

Class Name: CrcController	
Responsibilities: ▪ Create crc cards ▪ Save new crc cards ▪ Delete a crc card ▪ View a crc card ▪ Update a crc field ▪ Link a Use case to a Crc card ▪ Unlink a Use case from a Crc card	Collaborations: ▪ crcService ▪ useCaseService

Class Name: ProjectController	
Responsibilities: ▪ Display project dashboard ▪ Create a new project ▪ View project ▪ Delete a project ▪ View project diagram	Collaborations: ▪ projectService ▪ UsecaseService ▪ crsService ▪ diagramService

Class Name: UseCaseController	
Responsibilities: ▪ Create a new Use case ▪ Save Use case ▪ View Use case ▪ Delete a Use case ▪ Update a use case field ▪ Add an alternative flow ▪ Delete an alternative flow ▪ Link a Crc card to use case ▪ Unlink a Crc card from use case	Collaborations: ▪ UseCaseService ▪ alternativeFlowService ▪ crcService

Class Name: User Controller	
Responsibilities: ▪ Display user dashboard ▪ Display user profile ▪ Save user profile	Collaborations: ▪ UserService

Service (Interfaces)

Class Name: AlternativeFlowService	
Responsibilities: ▪ Find alternative flow by id ▪ Find alternative flow by use case id ▪ Add alternative flow ▪ Delete alternative flow	Collaborations: ▪ alternativeFlow

Class Name: CrcService	
Responsibilities: ▪ Find a Crc card by id ▪ Create a crc card ▪ Delete a crc card ▪ Update a crc field ▪ Link use case to CRC ▪ Unlink use case from CRC ▪ Get linked use cases ▪ Get linked CRCs	Collaborations: ▪ Crc ▪ Use case

Class Name: DiagramService	
Responsibilities: ▪ Generate Use Case diagram ▪ Generate Crc diagram ▪ Build the title of the project ▪ Build use case content ▪ Build crc content	Collaborations: ▪ Project ▪ UseCase ▪ Crc

Class Name: ProjectService	
Responsibilities: ▪ Create a new project ▪ Get projects from current user ▪ Delete project ▪ Find project by id	Collaborations: ▪ Project ▪ User

Class Name: UseCaseService	
Responsibilities:	Collaborations:

▪ Create a new use case ▪ Delete a use case ▪ Find use case by id ▪ Find use case by project id ▪ Update a use case field	▪ Project ▪ Usecase
---	--

Class Name: UserService	
Responsibilities: ▪ Save a user ▪ Check if user exists ▪ Load user by username ▪ Find user by username ▪ Update user credentials ▪ Get current user	Collaborations: ▪ User

Service Implementations

Class Name: AlternativeFlowImpl	
Responsibilities: ▪ Implements alternative flow operations logic	Collaborations: ▪ AlternativeFlowRepository ▪ USeCaseService

Class Name: CrcServiceImpl	
Responsibilities: ▪ Implement Crc card operations logic	Collaborations: ▪ CrcRepository ▪ projectService ▪ useCaseRepository

Class Name: DiagramServiceImpl	
Responsibilities: ▪ Implements diagram generation logic	Collaborations: ▪ projectService ▪ useCaseService ▪ crcService

Class Name: ProjectServiceImpl	
Responsibilities: ▪ Implements project operations logic	Collaborations: ▪ projectRepository ▪ userService

Class Name: UseCaseServiceImpl	
Responsibilities: ▪ Implements use case operations logic	Collaborations: ▪ useCaseRepository ▪ projectService

Class Name: UserServiceImpl	
Responsibilities: ▪ Implements user operations and authentication logic	Collaborations: ▪ bcryptPasswordEncoder ▪ UserRepository

Repositories

Class Name: AlternativeFlowRepository	
Responsibilities: ▪ Find alternative flow by id ▪ Find a list of alternative flows by use	Collaborations: ▪ AlternativeFlow ▪ UseCase

case id	
---------	--

Class Name: CrCRepository	
Responsibilities: - Find Crc card by id - Find crc card by project id	Collaborations: - Crc - Project

Class Name: ProjectRepository	
Responsibilities: - Find project by id - Find project by user	Collaborations: - Project - User

Class Name: UseCaseRepository	
Responsibilities: - Find use case by id - Find use case by project id	Collaborations: - UseCase - Project

Class Name: UserRepository	
Responsibilities: - Find user by id - Find user by username	Collaborations: User

Config

Class Name: CustomSecuritySuccessHandler	
Responsibilities: Handle successful authentication	Collaborations: Authentication

▪ Determine target URL ▪ Redirect user after login ▪ Check if response is committed ▪ Apply redirect strategy ▪ Extend Spring Security authentication success handling	▪ HttpServletRequest ▪ HttpServletResponse ▪ RedirectStrategy ▪ DefaultRedirectStrategy ▪ SimpleUrlAuthenticationSuccessHandler
--	---

Class Name: WebMvcConfig	
Responsibilities: ▪ Configure MVC settings ▪ Register view controllers ▪ Map URL paths to views ▪ Define the default homepage route	Collaborations: ▪ WebMvcConfigurer ▪ ViewControllerRegistry ▪ Spring MVC Framework ▪ View Controllers

Class Name: WebSecurityConfig	
Responsibilities: ▪ Configure application security ▪ Define authentication mechanism ▪ Configure password encryption ▪ Configure authentication provider ▪ Configure login and logout behavior ▪ Define authorization rules for URLs ▪ Register security beans	Collaborations: ▪ HttpSecurity ▪ UserServiceImpl ▪ BCryptPasswordEncoder ▪ CustomSecuritySuccessHandler ▪ SecurityFilterChain ▪ UserDetailsService ▪ DaoAuthenticationProvider

Domain Model

Class Name: AlternativeFlow	
Responsibilities: ▪ Store alternative flow data ▪ Maintain description	Collaborations: ▪ UseCase

▪ Associate with a use case	
---	--

Class Name: Crc	
Responsibilities: ▪ Store Crc card information ▪ Store class name ▪ Store responsibilities, collaborations ▪ Associate Crc card with one or more use cases ▪ Provide access to Crc data	Collaborations: ▪ Project ▪ UseCase

Class Name: Project	
Responsibilities: ▪ Store project data information ▪ Store project name ▪ Associate a project with a user ▪ Provide access to project data	Collaborations: ▪ User

Class Name: UseCase	
Responsibilities: ▪ Store use case information ▪ Store actor information ▪ Store preconditions, main flow, post conditions ▪ Associate a use case with a project ▪ Associate a use case with crc cards	Collaborations: ▪ Project ▪ Crc

Class Name: User	
Responsibilities: ▪ Store user credentials and personal	Collaborations:

data ▪ Provide username for authentication ▪ Support Spring Security authentication	▪ UserDetails ▪ GrantedAuthority
--	--